WISE ROUND 3: SPEND SQUAD — VIPER EDITION

All 6 Scenarios in Descriptive + Code + What to SAY Format

Card Product: ordering, transacting, 3DS, bill split, recurring, disputes

---

HOW THIS ROUND WORKS

From the candidate pack: "*Hypothetically existing feature in a mobile app — identify how it's built, then proposed improvements*."

Your JD is **Spend Squad** — card product processing **£26 billion annually** across **40+ countries**. The feature will be card-related.

The Format:

0:15 - 0:20	They describe the feature
0:20 - 0:22	You clarify requirements
0:22 - 0:25	You explain user journey (START HERE — product thinking)
0:25 - 0:40	You draw + explain VIPER architecture
0:40 - 0:50	They add requirements — you extend
0:50 - 0:55	Testing, production, trade-offs
0:55 - 1:00	Your questions

For EVERY Scenario — This Is Your Opening:

> "Let me start with what the customer experiences, then I'll design the architecture that makes it happen. I'll use VIPER since I know Wise uses it — View, Presenter, Interactor, Router, Entity. Each component has one job, and the whole module is independently testable."

SCENARIO 1: CARD TRANSACTION & REAL-TIME NOTIFICATION

★★★★★ Most Likely — Core of Spend Squad

Their Prompt:

> "When a customer pays with their Wise card, they get an instant notification showing the amount, merchant, and currency. How would you architect this on mobile?"

Your Clarifying Questions:

> "How quickly should the notification appear — sub-second from card tap?"
> "Should the notification look different when the app is in foreground versus background?"
> "Is the amount always final, or can it change — like restaurants with tips?"
> "Should I handle multi-currency — card charged in EUR but balance is SGD?"

User Journey (Start Here — Product Thinking):

> "The customer taps their Wise card at a coffee shop in Paris. Within one second, their phone buzzes with a notification: '💳 Starbucks Paris — €4.50 (S$6.53 from your SGD balance)'. If the app is open, they see an in-app banner with haptic feedback. The transaction appears at the top of their transaction list. Their SGD balance updates to reflect the deduction. Let me design the architecture that makes this happen."

High-Level Flow:

```
Card tap at POS
  → Visa/Mastercard network
  → Wise Payment Processor
  → Kafka event: card.transaction.created
  → Two consumers in parallel:
    ├── Transaction Service → stores in PostgreSQL
    └── Notification Service → APNs push to mobile
  → Mobile receives push
  → Shows notification + updates transaction list + updates balance
```

What to SAY while drawing this:

> "The backend publishes a single Kafka event that two services consume independently. The notification service doesn't wait for the transaction service to finish storing — they run in parallel. This means the push notification can arrive before the transaction is even written to the database. That's why I do an optimistic insert on mobile from the push data, then confirm with an API refresh."

VIPER Architecture:

```
TransactionListModule (VIPER)
│
├── VIEW (Protocol: TransactionListViewProtocol)
│   What it does: Renders transaction list grouped by date.
│   Shows loading/error/empty states. Pull-to-refresh.
│   Handles foreground notification banner with haptic.
│   Completely dumb — zero business logic.
│
├── PRESENTER (TransactionListPresenter)
│   What it does: Receives push data from View.
│   Asks Interactor to process the new transaction.
```

```
│  Formats transactions for display:
│    Decimal → "€4.50" with correct currency symbol
│    Date → "Today, 2:30 PM" or "Yesterday"
│    Status → pending icon, completed checkmark, declined X
│  Tells Router to navigate on tap.
│
├── INTERACTOR (TransactionListInteractor)
│  What it does: Business logic layer.
│  Cache-first: show cached transactions immediately.
│  Refresh from API in background.
│  Handle push: parse, insert locally, trigger balance refresh.
│  Filter by category/date/status.
│
├── ROUTER (TransactionListRouter)
│  What it does: Navigation only.
│  Tap transaction → push TransactionDetailModule
│  Tap dispute → present DisputeFlowModule modally
│  Deep link from push → navigate to specific transaction
│
├── ENTITY
│  CardTransaction, TransactionStatus, MultiCurrencyAmount
│
└── REPOSITORY (called by Interactor)
   ├── APIClient: GET /v3/spend/profiles/{id}/cards/transactions
   └── LocalCache: Encrypted CoreData for offline access
```

Code:

```swift
```

```swift
// ═══ ENTITY ═══

struct CardTransaction: Codable, Identifiable, Equatable {
    let id: String
    let merchantName: String
    let merchantCategory: String
    let amount: Decimal            // What merchant charged
    let currency: String           // Currency merchant charged in
    let convertedAmount: Decimal?  // What was deducted from balance
    let balanceCurrency: String?   // User's balance currency
    let rate: Decimal?             // FX rate if multi-currency
    let status: TransactionStatus
    let timestamp: Date
}

enum TransactionStatus: String, Codable {
    case pending    // Authorization hold — amount might change
    case completed  // Settled — final amount
    case declined   // Rejected by card network or spend controls
    case reversed   // Refunded by merchant
}

// ═══ VIEW PROTOCOL ═══

protocol TransactionListViewProtocol: AnyObject {
    func showTransactions(_ viewModels: [TransactionCellViewModel])
    func showLoading()
    func hideLoading()
    func showError(_ message: String)
    func showEmpty(_ message: String)
    func showInAppNotification(_ viewModel: TransactionCellViewModel)
    func updateBalance(_ formatted: String)
}

struct TransactionCellViewModel {
    let id: String
    let merchantName: String
    let primaryAmount: String      // "€4.50"
    let secondaryAmount: String?   // "S$6.53 from SGD" — if multi-currency
    let rateInfo: String?          // "Rate: 1 EUR = 1.45 SGD"
    let statusIcon: String         // "checkmark.circle" or "clock" or "xmark"
    let categoryIcon: String       // "cup.and.saucer" for coffee
    let timeAgo: String            // "2 min ago" or "Yesterday, 3:15 PM"
    let isPending: Bool            // Show "Pending" badge
}
```

```
// ═══ PRESENTER ═══

class TransactionListPresenter: TransactionListPresenterProtocol {
    weak var view: TransactionListViewProtocol?
    var interactor: TransactionListInteractorProtocol
    var router: TransactionListRouterProtocol

    private let formatter = CurrencyFormatter()
    private let dateFormatter = RelativeDateFormatter()

    func viewDidAppear() {
        view?.showLoading()
        interactor.fetchTransactions()
    }

    func didReceivePushNotification(_ data: [String: Any]) {
        interactor.handlePushTransaction(data)
    }

    func didSelectTransaction(_ id: String) {
        guard let transaction = interactor.transaction(byId: id) else { return }
        router.navigateToDetail(transaction)
    }

    func didTapDispute(_ id: String) {
        guard let transaction = interactor.transaction(byId: id) else { return }
        router.navigateToDispute(transaction)
    }

    func didPullToRefresh() {
        interactor.fetchTransactions()
    }
}

extension TransactionListPresenter: TransactionListInteractorOutput {

    func didFetchTransactions(_ transactions: [CardTransaction]) {
        view?.hideLoading()

        if transactions.isEmpty {
            view?.showEmpty("No transactions yet. Use your card to get started!")
            return
        }

        let viewModels = transactions.map { formatTransaction($0) }
        view?.showTransactions(viewModels)
    }
```

```
    func didReceiveNewTransaction(_ transaction: CardTransaction) {
        let viewModel = formatTransaction(transaction)
        view?.showInAppNotification(viewModel) // Banner + haptic
    }

    func didFailFetchTransactions(_ error: Error) {
        view?.hideLoading()
        view?.showError("Unable to load transactions. Tap to retry.")
    }

    // ── Formatting ──

    private func formatTransaction(_ txn: CardTransaction) -> TransactionCellViewModel {
        let primary = formatter.format(txn.amount, currency: txn.currency)

        let secondary: String? = txn.convertedAmount.flatMap { converted in
            guard let balCurrency = txn.balanceCurrency else { return nil }
            return "\(formatter.format(converted, currency: balCurrency)) from \(balCurrency)"
        }

        let rateInfo: String? = txn.rate.flatMap { rate in
            guard let balCurrency = txn.balanceCurrency else { return nil }
            return "Rate: 1 \(txn.currency) = \(rate) \(balCurrency)"
        }

        return TransactionCellViewModel(
            id: txn.id,
            merchantName: txn.merchantName,
            primaryAmount: primary,
            secondaryAmount: secondary,
            rateInfo: rateInfo,
            statusIcon: iconFor(txn.status),
            categoryIcon: categoryIcon(txn.merchantCategory),
            timeAgo: dateFormatter.format(txn.timestamp),
            isPending: txn.status == .pending
        )
    }
}

// ═══ INTERACTOR ═══

class TransactionListInteractor: TransactionListInteractorProtocol {
    weak var output: TransactionListInteractorOutput?
    private let repository: TransactionRepositoryProtocol
    private let balanceService: BalanceServiceProtocol
    private var allTransactions: [CardTransaction] = []
```

```
    func fetchTransactions() {
        // Phase 1: Show cached immediately
        if let cached = repository.getCached() {
            allTransactions = cached
            output?.didFetchTransactions(cached)
        }

        // Phase 2: Refresh from network
        Task {
            do {
                let fresh = try await repository.fetchFromServer()
                allTransactions = fresh
                await MainActor.run { output?.didFetchTransactions(fresh) }
            } catch {
                await MainActor.run {
                    if allTransactions.isEmpty { // Only show error if no cached data
                        output?.didFailFetchTransactions(error)
                    }
                }
            }
        }
    }

    func handlePushTransaction(_ data: [String: Any]) {
        guard let txn = parseTransactionFromPush(data) else { return }

        // Optimistic insert — show immediately
        allTransactions.insert(txn, at: 0)
        repository.insertLocal(txn)
        output?.didReceiveNewTransaction(txn)

        // Refresh balance
        Task { await balanceService.refresh() }

        // Confirm with API (push might have preliminary data)
        Task {
            try? await Task.sleep(nanoseconds: 2_000_000_000) // Wait 2s for backend to settle
            let _ = try? await repository.fetchFromServer()
        }
    }

    func transaction(byId id: String) -> CardTransaction? {
        allTransactions.first { $0.id == id }
    }
}
```

```swift
// ═══ ROUTER ═══

class TransactionListRouter: TransactionListRouterProtocol {
    weak var viewController: UIViewController?

    func navigateToDetail(_ transaction: CardTransaction) {
        let detailModule = TransactionDetailBuilder.build(transaction: transaction)
        viewController?.navigationController?.pushViewController(detailModule, animated: true)
    }

    func navigateToDispute(_ transaction: CardTransaction) {
        let disputeModule = DisputeFlowBuilder.build(transaction: transaction)
        viewController?.present(disputeModule, animated: true)
    }
}
```

Multi-Currency — The Hard Part:

When a customer pays €4.50 in Paris but their balance is SGD, we need to show both amounts. This is the exact same problem as BFX at PayPal — determining the conversion rate, calculating the fee, and displaying the deducted amount in the user's balance currency.

```swift
struct MultiCurrencyDisplay {
    let chargedAmount: Decimal      // €4.50 — what merchant charged
    let chargedCurrency: String     // EUR
    let deductedAmount: Decimal     // S$6.53 — what left the balance
    let deductedCurrency: String    // SGD
    let rate: Decimal               // 1.45

    // Display:
    // Primary:   "€4.50 at Starbucks Paris"
    // Secondary: "S$6.53 from your SGD balance"
    // Detail:    "Rate: 1 EUR = 1.45 SGD"
}
```

What to SAY:

> "The primary amount shows what the merchant charged in their currency. The secondary shows what was deducted from the user's balance — this is what the customer actually cares about. The rate is shown for transparency, which is core to Wise's mission. This is exactly what I work on at PayPal with BFX — dual-currency display with the conversion rate and fee visible."

Push Notification Architecture:

The notification handling differs based on whether the app is in foreground or background. Both paths need to

update the local transaction list and balance.

```swift
// ═══ NOTIFICATION HANDLER ═══

class CardNotificationHandler {
    private let transactionStore: TransactionStoreProtocol
    private let balanceService: BalanceServiceProtocol

    // BACKGROUND: Rich notification via UNNotificationServiceExtension
    // Downloads merchant logo, formats: "💳 Starbucks — €4.50 (S$6.53)"
    // When user taps → deep link to transaction detail

    // FOREGROUND: In-app banner
    func handleForegroundNotification(_ userInfo: [AnyHashable: Any]) {
        guard let txnData = userInfo["transaction"] as? [String: Any],
              let txn = parseTransaction(txnData) else { return }

        // 1. Optimistic insert into local list
        transactionStore.insert(txn)

        // 2. Show in-app banner with haptic
        HapticManager.notification(.success)
        BannerPresenter.show(
            title: txn.merchantName,
            subtitle: "\(txn.currency) \(txn.amount)",
            duration: 5.0,
            onTap: { Router.navigateToTransaction(txn.id) }
        )

        // 3. Refresh balance
        Task { await balanceService.refresh() }
    }
}
```

What to SAY:

> "For background notifications, I use a UNNotificationServiceExtension to download the merchant logo and format a rich notification before iOS displays it. The user sees the merchant name and amount without opening the app.
> For foreground notifications, I show an in-app banner with haptic feedback — more native-feeling than a system notification when the user is already in the app. Both paths optimistically insert the transaction into the local list and refresh the balance."

Trade-offs:

Push vs Polling:

> "Push for real-time transaction alerts, absolutely. Polling every second would drain the battery and still have up to 1 second delay. Push is event-driven — the server sends it only when something happens. But I'd add pull-to-refresh as a fallback in case a push is delayed or lost."

Optimistic insert vs Wait for API:

> "I show the transaction from push data immediately — the user expects instant feedback when they tap their card. The trade-off is the push might have preliminary data. For example, at a restaurant, the authorization amount doesn't include the tip. So I show 'Pending' status and the authorization amount, then update when the settlement comes through. This is a standard two-phase pattern in card processing."

Their Follow-Up — "Now add spending categories and monthly summary":

I'd create a new VIPER module — SpendingSummaryModule — rather than bloating the transaction list. The backend assigns categories per transaction using merchant category codes. The mobile fetches a summary API and renders a donut chart.

```swift
// New VIPER Module — doesn't modify TransactionList module

class SpendingSummaryInteractor: SpendingSummaryInteractorProtocol {
    func fetchSummary(month: String) {
        Task {
            // GET /v3/spend/profiles/{id}/summary?month=2026-02
            let summary = try await apiClient.fetchSpendingSummary(month: month)
            await MainActor.run { output?.didFetchSummary(summary) }
        }
    }
}

struct SpendingCategory: Identifiable {
    let id: String
    let name: String          // "Food & Drink"
    let total: Decimal        // S$342.50
    let percentage: Double    // 28%
    let transactionCount: Int // 15 transactions
    let icon: String          // "fork.knife"
}

// Presenter formats for display:
// "Food & Drink — S$342.50 (28%)" with donut chart segment
// Tap category → filtered transaction list (Router handles)
// Month-over-month: "15% more than January"
```

What to SAY:

> "New VIPER module, not added to the existing transaction list — Single Responsibility. The backend assigns categories using merchant category codes, so the mobile just displays what the backend provides. I'd use Swift Charts for the donut chart since it's iOS 16+ native. Tapping a category segment navigates to a filtered transaction list — the Router handles this navigation."

SCENARIO 2: 3DS AUTHENTICATION

★★★★ — In Your JD + Wise Has Public 3DS API

Their Prompt:

> "When a customer makes an online purchase, sometimes 3D Secure authentication is required. How would you architect the mobile 3DS flow?"

User Journey:

> "The customer uses their Wise card on Amazon. Amazon triggers 3DS verification. Wise's backend receives the challenge and sends a push notification to the customer's phone: 'Approve S$150 at Amazon?' The customer opens the notification, sees the merchant and amount, authenticates with Face ID, and taps Approve. The app sends the result to Wise's backend within the 5-minute window. Amazon's checkout completes."

How 3DS Works at Wise (from their actual API docs):

```
1. Card used online → Merchant requests 3DS
2. Card network → Wise backend receives challenge
3. Wise backend → Push notification to mobile app
4. Mobile shows: "Approve S$150 at Amazon?" with countdown timer
5. User authenticates with Face ID → taps Approve
6. App → POST /v3/spend/profiles/{id}/3dsecure/challenge-result
7. Wise → Card network → Merchant checkout completes

CRITICAL: Must respond within 5 minutes or challenge expires.
Wise's API: "Only the first call will be processed, duplicates ignored."
```

VIPER Architecture:

```
ThreeDSModule (VIPER)
│
├── VIEW
```

```
│  Modal sheet over current screen.
│  Shows: merchant name, amount, currency.
│  Countdown timer (red when < 60 seconds).
│  Approve / Decline buttons.
│  Loading state while submitting.
│  Expired state if timer runs out.
│
├── PRESENTER
│  Formats amount with currency symbol.
│  Manages countdown timer display.
│  Triggers color change at 60 seconds.
│  Handles expired state — invalidates UI.
│  Delegates approve/decline to Interactor.
│
├── INTERACTOR
│  Validates challenge not expired (client-side check).
│  Requires biometric authentication before submitting.
│  Calls API: POST /v3/spend/profiles/{id}/3dsecure/challenge-result
│  Handles: success, failure, expiry, biometric cancel.
│
├── ROUTER
│  Presented modally from push notification deep link.
│  Dismiss on completion.
│  Navigate to transaction detail after approval.
│
└── ENTITY
   ThreeDSChallenge, ChallengeResult
```

Code:

```swift
```

```
// ═══ ENTITY ═══

struct ThreeDSChallenge: Codable {
    let transactionReference: String
    let merchantName: String
    let amount: Decimal
    let currency: String
    let expiresAt: Date

    var isExpired: Bool { Date() > expiresAt }
    var secondsRemaining: Int { max(0, Int(expiresAt.timeIntervalSinceNow)) }
}

enum ChallengeResult: String, Codable {
    case approved = "APPROVED"
    case declined = "DECLINED"
}

// ═══ INTERACTOR ═══

class ThreeDSInteractor: ThreeDSInteractorProtocol {
    weak var output: ThreeDSInteractorOutput?
    private let apiClient: SpendAPIClientProtocol
    private let biometricAuth: BiometricAuthProtocol

    func approve(challenge: ThreeDSChallenge) {
        Task {
            // Step 1: Check not expired
            guard !challenge.isExpired else {
                await MainActor.run { output?.didExpire() }
                return
            }

            // Step 2: Biometric auth — stolen phone can't approve
            do {
                let authenticated = try await biometricAuth.authenticate(
                    reason: "Approve \(challenge.currency) \(challenge.amount) at \(challenge.merchantName)"
                )
                guard authenticated else {
                    await MainActor.run { output?.didCancelAuth() }
                    return
                }
            } catch {
                await MainActor.run { output?.didFailAuth(error) }
                return
            }
```

```
        // Step 3: Submit result to Wise API
        do {
            try await apiClient.submit3DSResult(
                transactionReference: challenge.transactionReference,
                status: .approved
            )
            await MainActor.run { output?.didSubmitResult(.approved) }
        } catch {
            await MainActor.run { output?.didFailSubmission(error) }
        }
    }
}

func decline(challenge: ThreeDSChallenge) {
    Task {
        try? await apiClient.submit3DSResult(
            transactionReference: challenge.transactionReference,
            status: .declined
        )
        await MainActor.run { output?.didSubmitResult(.declined) }
    }
}
}

// ═══ PRESENTER ═══

class ThreeDSPresenter: ThreeDSPresenterProtocol {
    weak var view: ThreeDSViewProtocol?
    var interactor: ThreeDSInteractorProtocol
    var router: ThreeDSRouterProtocol

    private var countdownTimer: Timer?
    private let formatter = CurrencyFormatter()

    func viewDidAppear(challenge: ThreeDSChallenge) {
        // Format display
        view?.showChallenge(
            merchant: challenge.merchantName,
            amount: formatter.format(challenge.amount, currency: challenge.currency)
        )

        // Start countdown
        startCountdown(expiresAt: challenge.expiresAt)
    }

    func didTapApprove(challenge: ThreeDSChallenge) {
```

```
        view?.showSubmitting()
        interactor.approve(challenge: challenge)
    }

    func didTapDecline(challenge: ThreeDSChallenge) {
        interactor.decline(challenge: challenge)
    }

    private func startCountdown(expiresAt: Date) {
        countdownTimer = Timer.scheduledTimer(withTimeInterval: 1, repeats: true) {
            [weak self] _ in
            let remaining = Int(expiresAt.timeIntervalSinceNow)
            if remaining <= 0 {
                self?.countdownTimer?.invalidate()
                self?.view?.showExpired()
            } else {
                self?.view?.updateCountdown(
                    seconds: remaining,
                    isUrgent: remaining < 60  // Red when under 60s
                )
            }
        }
    }
}

extension ThreeDSPresenter: ThreeDSInteractorOutput {
    func didSubmitResult(_ result: ChallengeResult) {
        countdownTimer?.invalidate()
        view?.showSuccess(result == .approved ? "Payment approved" : "Payment declined")
        router.dismissAfterDelay(seconds: 2)
    }

    func didExpire() {
        countdownTimer?.invalidate()
        view?.showExpired()
    }

    func didCancelAuth() {
        view?.hideSubmitting()
        // User cancelled Face ID — stay on screen, let them try again
    }

    func didFailSubmission(_ error: Error) {
        view?.showError("Failed to send response. Tap to retry.")
    }
}
```

Key Concerns (Mention All 5):

Timing:

> "The 5-minute window is non-negotiable — Wise's API returns 400 after expiry. The countdown timer must be prominent. I change the color to red under 60 seconds and increase the font size. If the timer expires while the user is looking at the screen, I immediately hide the buttons and show 'Challenge expired — the merchant will need to retry.'"

Deep Linking:

> "The push notification must open the 3DS screen directly — even if the app was killed. I'd encode the challenge data in the push payload's userInfo and handle it in the app delegate's notification response handler. The Router presents the 3DS module modally over whatever screen is currently visible."

Biometric Auth:

> "Before submitting approval, I require Face ID or Touch ID. A stolen phone shouldn't be able to approve transactions. The biometric prompt shows the merchant and amount: 'Approve S$150 at Amazon'. If biometric fails, the user stays on the screen — they can try again or decline."

Idempotency:

> "Wise's API processes only the first call — duplicates are ignored but return success. This means if the user taps Approve twice rapidly, or if the network retries the request, we won't get an error. I still disable the button after first tap to prevent confusion."

Fallback:

> "If the push doesn't arrive — phone offline, notifications disabled — Wise falls back to SMS OTP. The mobile app should handle both paths: push-based in-app approval and OTP entry screen."

SCENARIO 3: CARD ORDERING

★★★★ — In Your JD

Their Prompt:

> "Walk us through how you'd architect the card ordering experience on mobile."

User Journey:

> "Two paths diverge here. Virtual card: user taps 'Get a card', selects Virtual, reviews the free card details, confirms with Face ID, and immediately sees their card number with an 'Add to Apple Wallet' button. They can use it for online purchases within 30 seconds.
> Physical card: user selects Physical, enters their delivery address through a Dynamic Form that varies by country, reviews the order with delivery estimate and fee, confirms with Face ID, and sees a tracking

> screen: Ordered → Printed → Shipped → Delivered. Push notifications at each stage."

VIPER with Coordinator (Multi-Step Flow):

For multi-step flows, I use a Coordinator pattern on top of VIPER. Each step is an independent VIPER module. The Coordinator manages shared state across steps and controls the flow direction.

```swift
```

```
// ═══ COORDINATOR — manages the multi-step flow ═══

class CardOrderCoordinator {
    weak var delegate: CardOrderCoordinatorDelegate?
    private var navigationController: UINavigationController

    // Shared state across steps
    private var selectedCardType: CardType?
    private var selectedVariant: CardVariant?
    private var deliveryAddress: DeliveryAddress?

    func start() {
        let cardTypeModule = CardTypeBuilder.build(delegate: self)
        navigationController.pushViewController(cardTypeModule, animated: true)
    }

    func didSelectCardType(_ type: CardType, variant: CardVariant) {
        selectedCardType = type
        selectedVariant = variant

        if type == .virtual {
            showReview()   // Skip address — virtual doesn't need it
        } else {
            showDeliveryAddress()
        }
    }

    func didEnterAddress(_ address: DeliveryAddress) {
        deliveryAddress = address
        showReview()
    }

    func didConfirmOrder() {
        submitOrder()
    }

    private func showDeliveryAddress() {
        // Dynamic Forms — backend sends form structure per country
        let country = getUserCountry()
        let module = DeliveryAddressBuilder.build(country: country, delegate: self)
        navigationController.pushViewController(module, animated: true)
    }

    private func showReview() {
        let summary = OrderSummary(
            cardType: selectedCardType!,
```

```swift
            variant: selectedVariant!,
            address: deliveryAddress,
            fee: selectedCardType == .virtual ? Decimal.zero : Decimal(string: "5.00")!,
            estimatedDelivery: selectedCardType == .physical ? estimateDelivery() : nil
        )
        let module = ReviewOrderBuilder.build(summary: summary, delegate: self)
        navigationController.pushViewController(module, animated: true)
    }

    private func submitOrder() {
        let request = CardOrderRequest(
            cardType: selectedCardType!,
            variant: selectedVariant!,
            deliveryAddress: deliveryAddress,
            idempotencyKey: UUID().uuidString  // Prevent duplicate card orders
        )
        // Interactor handles: biometric auth → API call → show result
    }
}
```

What to SAY:

> "I use a Coordinator because the flow has conditional steps — virtual cards skip the address step. Each step is an independent VIPER module that doesn't know about other steps. The Coordinator holds shared state and controls the flow. This means I can reorder steps, skip steps, or add new steps without modifying any existing module.
> The address step uses Dynamic Forms — Wise's server-driven UI framework. The backend sends the form structure per country. Different countries need different fields — UK needs a postcode, Germany needs a PLZ, India needs a PIN code. The mobile just renders whatever the backend sends. No app update needed when regulatory requirements change."

Apple Wallet Integration:

For virtual cards, I'd use PassKit to add the card to Apple Wallet. This gives customers contactless payment within 30 seconds of ordering.

```swift
```

```swift
import PassKit

class AppleWalletInteractor {
    func addToWallet(cardDetails: VirtualCardDetails) async throws {
        guard PKPassLibrary.isPassLibraryAvailable(),
              PKAddPaymentPassViewController.canAddPaymentPass() else {
            throw CardError.walletNotAvailable
        }

        // Wise backend provides encrypted card data
        // PassKit handles provisioning to Apple Wallet
        // The card appears in Wallet app for NFC payments
    }
}
```

What to SAY:

> "Virtual card is the magic moment — the user orders a card and can use it within 30 seconds via Apple Pay. I'd use PassKit's In-App Provisioning API. Wise's backend provides the encrypted card data, PassKit handles the rest. For physical cards, Apple Wallet shows the card for tap-to-pay after activation."

Physical Card Tracking:

```swift
```

```swift
class CardTrackingInteractor: CardTrackingInteractorProtocol {
    private var pollingTask: Task<Void, Never>?

    func startTracking(cardId: String) {
        pollingTask = Task {
            while !Task.isCancelled {
                let status = try? await apiClient.getCardStatus(cardId)
                if let status = status {
                    await MainActor.run { output?.didUpdateStatus(status) }
                    if status == .delivered || status == .activated { break }
                }
                try? await Task.sleep(nanoseconds: 60_000_000_000) // Every 60s
            }
        }
    }

    func stopTracking() { pollingTask?.cancel() }
}

// Status timeline: Ordered → Printed → Shipped → Delivered → Activated
// Push notification at each transition
// Activation: enter last 4 digits or scan card with camera
```

SCENARIO 4: BILL SPLIT

★★★ — In Your JD

Their Prompt:

> "After a restaurant payment, the user can split the bill with friends. How would you design this?"

User Journey:

> "The customer just paid S$120 at a restaurant. They tap the transaction and see 'Split this bill'. They add 3 friends — two are Wise users found by phone number, one isn't. They choose equal split. The app shows S$30 each, with the last friend paying S$30.00 (no rounding issue with 4 people and S$120). They confirm, and Wise sends push notifications to the two Wise friends and an SMS link to the non-Wise friend. The customer sees a progress tracker: '1 of 3 friends have paid.'"

The Rounding Problem (Critical Fintech Detail):

This is the hardest part of bill splitting and the thing that separates fintech engineers from general mobile engineers. You can never create or lose money through rounding.

```swift
class BillSplitCalculator {

    func splitEqually(amount: Decimal, participantCount: Int) -> [Decimal] {
        guard participantCount > 0 else { return [] }

        let rawShare = amount / Decimal(participantCount)
        let roundedShare = roundToTwoDecimals(rawShare)

        // N-1 people pay the rounded amount
        let othersTotal = roundedShare * Decimal(participantCount - 1)

        // Last person pays the remainder — absorbs rounding
        let lastPersonShare = amount - othersTotal

        var shares = Array(repeating: roundedShare, count: participantCount - 1)
        shares.append(lastPersonShare)
        return shares

        // S$100 ÷ 3: [33.33, 33.33, 33.34] → total = 100.00 ✅
        // S$100 ÷ 7: [14.29 × 6 = 85.74] + [14.26] = 100.00 ✅
    }

    private func roundToTwoDecimals(_ value: Decimal) -> Decimal {
        var result = value
        var rounded = Decimal()
        NSDecimalRound(&rounded, &result, 2, .bankers)
        return rounded
    }
}
```

What to SAY:

> "The fintech rule: never create or lose money through rounding. S$100 split 3 ways — naive approach gives 33.33 × 3 = 99.99, we've lost a cent. My approach: N-1 people pay the rounded share, the last person pays the remainder. Total is always exactly the original amount. I use banker's rounding — NSDecimalRound with .bankers — which is standard in financial software."

Wise vs Non-Wise Users:

```swift
```

```swift
// Two completely different payment flows

struct Participant: Identifiable, Codable {
    let id: String
    let name: String
    let contact: String          // Phone or email
    let isWiseUser: Bool
    let wiseProfileId: String?
    var amount: Decimal
    var paymentStatus: PaymentStatus // .pending, .paid, .declined
}

// PRESENTER formats differently:
// Wise user:     "John (Wise) — S$33.33 · Free & instant"
// Non-Wise user: "Alice (SMS)  — S$33.33 · Via payment link"
```

What to SAY:

> "Two very different flows. Wise-to-Wise is a balance transfer — free, instant, the friend gets a push notification and pays with one tap. Non-Wise gets an SMS or WhatsApp link to a payment page where they pay by card — this has a processing fee. The Presenter should display this difference upfront so the user knows."

SCENARIO 5: RECURRING PAYMENTS / SUBSCRIPTIONS

★★★ — In Your JD

Their Prompt:

> "Users want to see and manage their recurring card payments. How would you design this?"

Architecture:

The tricky part here isn't the mobile — it's the detection. The backend identifies recurring patterns by analyzing transaction history for the same merchant at regular intervals with similar amounts. The mobile just displays what the backend has identified and lets the user manage them.

```swift
```

```swift
// ═══ ENTITY ═══

struct RecurringPayment: Identifiable, Codable {
    let id: String
    let merchantName: String
    let merchantLogo: URL?
    let amount: Decimal
    let currency: String
    let frequency: Frequency
    let nextExpectedDate: Date
    let lastChargeDate: Date
    let isActive: Bool

    enum Frequency: String, Codable {
        case weekly, monthly, quarterly, yearly
    }
}

// ═══ INTERACTOR ═══

class RecurringPaymentsInteractor: RecurringPaymentsInteractorProtocol {
    weak var output: RecurringPaymentsInteractorOutput?
    private let apiClient: SpendAPIClientProtocol

    func fetchRecurringPayments() {
        Task {
            let payments = try await apiClient.fetchRecurringPayments()
            let totalMonthly = calculateMonthlyTotal(payments)
            await MainActor.run {
                output?.didFetchPayments(payments, totalMonthly: totalMonthly)
            }
        }
    }

    func blockMerchant(_ merchantId: String) {
        Task {
            // POST /v3/spend/profiles/{id}/cards/{cardId}/controls
            // Add merchant to blocked list — prevents future charges
            try await apiClient.addSpendControl(
                type: .blockMerchant(merchantId)
            )
            await MainActor.run { output?.didBlockMerchant(merchantId) }
        }
    }

    private func calculateMonthlyTotal(_ payments: [RecurringPayment]) -> Decimal {
```

```
        payments
            .filter { $0.isActive }
            .map { normalizeToMonthly($0) }
            .reduce(Decimal.zero, +)
    }

    private func normalizeToMonthly(_ payment: RecurringPayment) -> Decimal {
        switch payment.frequency {
        case .weekly: return payment.amount * 4
        case .monthly: return payment.amount
        case .quarterly: return payment.amount / 3
        case .yearly: return payment.amount / 12
        }
    }
}

// PRESENTER formats:
// "You spend S$89.97/month on subscriptions"
// Netflix S$16.98/mo
// Spotify S$9.99/mo
// Gym S$63.00/mo
// "Next charge: Netflix on March 5"
// Tap → options: view transactions, block merchant, set alert before charge
```

What to SAY:

> "The mobile's job is display and management — detection is the backend's responsibility. For management, the key action is blocking a merchant through Wise's spend controls API. This prevents future charges from that merchant across all the user's cards. I'd also add a 'Notify before charge' toggle — push notification 1 day before each expected charge so the user can cancel the subscription or ensure sufficient balance."

SCENARIO 6: DISPUTE / CHARGEBACK

★★★ — In Your JD

Their Prompt:

> "A customer doesn't recognize a card transaction and wants to dispute it. How would you design the dispute flow?"

VIPER with Coordinator (Multi-Step Flow):

```
DisputeFlowCoordinator
```

```
│
├── Step 1: SelectReasonModule
│   Interactor checks eligibility:
│     Transaction completed? (not pending)
│     Within 120 days? (regulatory deadline)
│     Not already disputed?
│   7 reasons: unrecognized, wrong amount, duplicate,
│     not received, defective, cancelled, fraud
│   Fraud → immediately suggest card freeze
│
├── Step 2: ProvideDetailsModule
│   Text description field
│   Photo evidence upload:
│     Compress to max 2MB
│     Upload with progress indicator
│     Resumable on network interruption
│   Expected resolution amount (for wrong amount disputes)
│
├── Step 3: ReviewDisputeModule
│   Summary of claim
│   "Provisional credit may be applied within 48 hours"
│   Biometric auth before submitting
│   Idempotency key to prevent duplicate disputes
│
└── Step 4: DisputeTrackingModule
    Timeline: Filed → Under Review → With Card Network → Resolved
    Deadline warning: "45 days remaining" (red when < 7 days)
    Push notifications on status changes
    "More info needed" → deep link back to provide details
```

Key Code — Evidence Upload:

```swift
```

```swift
class DisputeEvidenceInteractor: DisputeEvidenceInteractorProtocol {
    weak var output: DisputeEvidenceInteractorOutput?
    private let apiClient: SpendAPIClientProtocol
    private let imageCompressor: ImageCompressorProtocol

    func uploadEvidence(photos: [UIImage], disputeId: String) {
        Task {
            for (index, photo) in photos.enumerated() {
                do {
                    // Step 1: Compress — phones take 12MP but receipts don't need that
                    let compressed = try imageCompressor.compress(photo, maxSizeKB: 2048)

                    // Step 2: Get pre-signed upload URL from backend
                    let uploadURL = try await apiClient.getEvidenceUploadURL(disputeId: disputeId)

                    // Step 3: Upload with progress tracking
                    try await apiClient.uploadFile(data: compressed, to: uploadURL) { progress in
                        Task { @MainActor in
                            self.output?.didUpdateUploadProgress(index: index, progress: progress)
                        }
                    }

                    await MainActor.run { output?.didCompleteUpload(index: index) }
                } catch {
                    await MainActor.run { output?.didFailUpload(index: index, error: error) }
                }
            }
        }
    }
}
```

What to SAY:

> "Evidence upload is the trickiest mobile concern. I compress images before upload — phones take 12MP photos but receipt images don't need that resolution. Max 2MB at 70% quality. I upload with progress indication using a background URLSession so uploads continue even if the user switches screens. Pre-signed URLs from the backend mean the mobile uploads directly to S3 without going through the API server.
> For fraud disputes, I immediately suggest freezing the card — don't wait for the dispute process. If there are multiple unrecognized transactions from the same merchant or timeframe, I'd offer bulk dispute: 'We found 3 suspicious transactions. Include them all?'"

CROSS-CUTTING CONCERNS — MENTION FOR ANY SCENARIO

Spend Controls (Unique to Card Product):

```swift
struct SpendControls: Codable {
    var contactlessLimit: Decimal?      // Max per contactless payment
    var onlinePaymentsEnabled: Bool     // Enable/disable online purchases
    var atmWithdrawalsEnabled: Bool     // Enable/disable ATM
    var magneticStripeEnabled: Bool     // Enable/disable swipe
    var monthlySpendLimit: Decimal?     // Total monthly cap
    var blockedMerchants: [String]      // Blocked merchant IDs
}
```

> "Spend controls are the card product's equivalent of feature flags for the user. When a transaction is declined due to spend controls, the notification should explain WHY: 'Online payments are disabled on your card. Enable them in Settings to complete this purchase.' Actionable error messages, not generic declines."

Card Freeze/Unfreeze:

> "Single tap with immediate effect. I'd use an optimistic UI update — show the card as frozen immediately, then confirm with the API. If the API call fails, revert and show an error. The backend must process this within milliseconds because the next card transaction could come any second after the freeze request."

Security (Always Mention):

> "Certificate pinning on all Spend API calls. Card details — number, CVV, expiry — stored in Keychain, never UserDefaults. Biometric auth before showing full card number, before approving 3DS, before confirming card order. Screen protection: blur the card details when app enters background so they're not visible in the app switcher."

BFX Connection (Your Advantage):

> "The Spend product maps directly to my BFX work at PayPal. BFX handles what happens when a Visa multi-currency card is used — the authorization, currency conversion decision, and fee calculation. I've worked on the exact same backend systems that power Wise's card authorization. The difference is Wise is the card issuer while PayPal is the processor — but the authorization flow, multi-currency logic, and spend controls architecture are fundamentally the same."

QUESTIONS TO ASK THEM (Pick 2)

1. "How does the mobile team handle Dynamic Forms for card features? Is the ordering flow server-driven?"
2. "What's the latency target for card transaction notifications — from card tap to push delivery?"

3. "How do you manage the VIPER to TCA migration? Screen-by-screen?"
4. "What's the biggest mobile technical challenge in the Spend product right now?"
5. "How do iOS and Android teams coordinate feature parity for card features?"